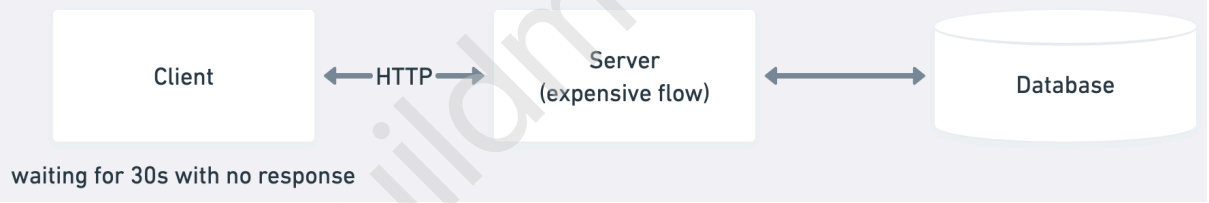
Manage Long Running Tasks

管理長時間執行任務的模式（Managing Long-Running Tasks pattern）把 API 請求拆成兩個階段：立即確認和背景處理。當用戶提交繁重的任務（例如影片編碼），web server 會立刻驗證請求、把一個 job 推進 queue（Redis / RabbitMQ），並回傳一個 job ID，這整個過程在毫秒內完成。與此同時，獨立的 worker process 持續輪詢 queue、抓取待處理的 job、執行真正耗時的工作，並在 database 裡更新 job 的狀態。

問題在哪裡

先從一個問題出發。想像你有一個簡單的網站，用戶可以查看他們的個人頁面。當用戶載入個人頁面時，你的 server 做一個快速的 database query 來抓取他們的資料。整個過程從查詢 database、格式化回應、回傳，總共不到 100 毫秒。用戶點一下，幾乎立刻就看到資訊。一切都很美好。

現在想像，不只是抓取資料，我們還需要產生一份用戶年度活動的 PDF 報告。這需要查詢多張表、在幾百萬筆資料中做聚合、渲染圖表、並產生一份格式化的文件。整個過程至少要 30 秒。



在同步處理（synchronous processing）的情況下，用戶的瀏覽器就這樣等了 30 秒。大多數 web server 和 load balancer 設有 30 到 60 秒的 timeout 限制，所以請求甚至可能來不及完成。就算完成了，用戶體驗也很差，因為他們盯著一個載入中的圖示，完全不知道進度如何。

PDF 報告不是特例。影片上傳需要幾分鐘的轉碼；個人照片上傳需要調整大小、裁切、產生多種縮圖尺寸；批量操作如發送電子報給幾千個用戶，或匯入大型 CSV 檔案，需要的時間更長。這些操作都遠超過用戶可以合理等待的範圍。

就算你能無限擴展，同步處理也提供了很糟糕的用戶體驗。用戶點下「產生報告」，然後.....什麼都沒有。它在運作嗎？失敗了嗎？該重新整理嗎？等了 30 秒後，很多用戶以為出了什麼問題，然後再試一次，製造出讓問題更嚴重的重複工作。

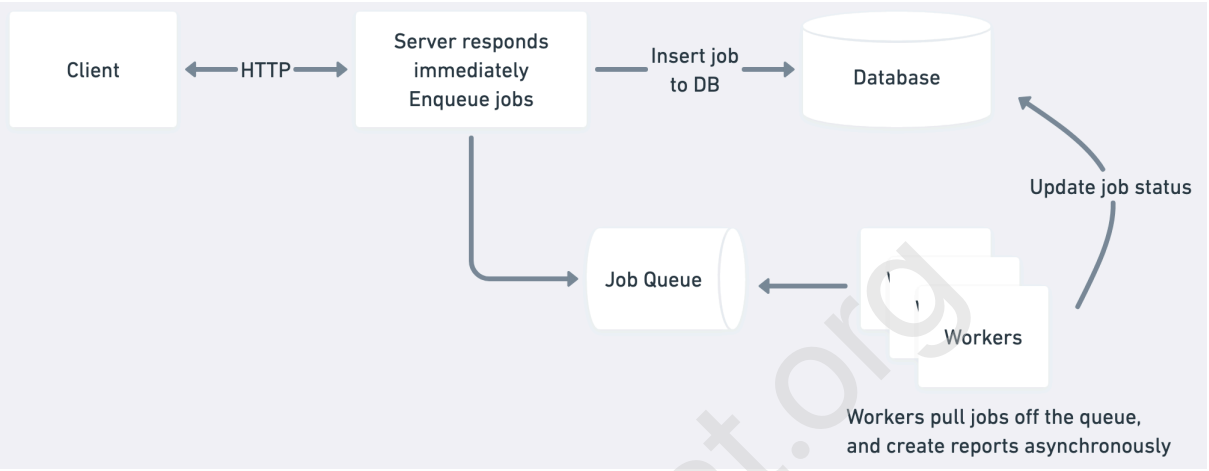
同步處理在操作超過幾秒時顯然有它的極限。替代方案是什麼？

buildmoat.org

解法的架構

不讓用戶等待我們產生 PDF，而是把操作拆成兩個部分。當他們點下「產生報告」，我們立刻把請求存進 queue，然後回傳一個回應：「我們正在產生你的報告，準備好了會通知你。」這只需要毫秒，不是幾分鐘。

Web server 的工作現在超級簡單：只需驗證請求、把它加進 queue、然後回傳一個 job ID。實際的 PDF 產生完全在別的地方進行。

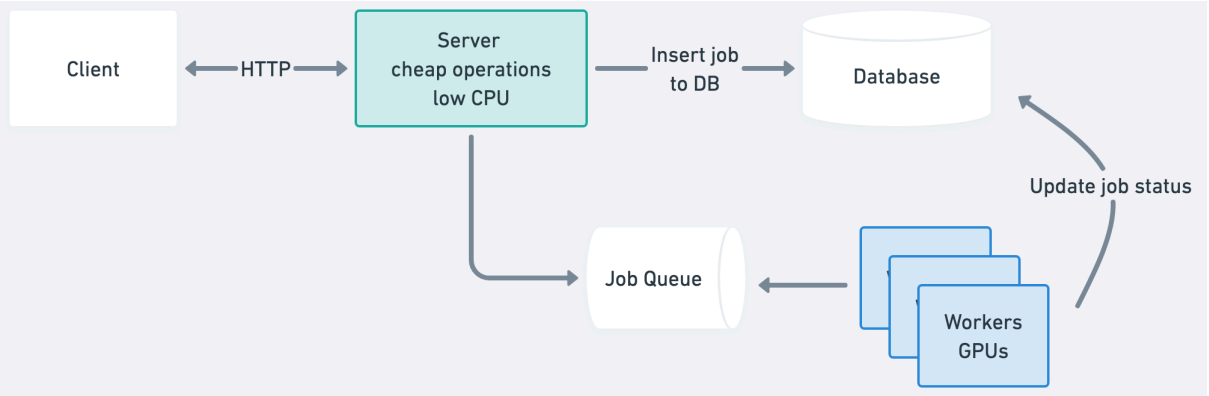


這就是管理長時間執行任務的模式（Managing Long-Running Tasks pattern）。核心概念是把**接受請求**和**處理請求**解耦。你的 web server 變成輕量級的請求路由器，快速確認工作並把它交接出去。一個獨立的 worker process 池（pool）負責真正繁重的工作。這些 worker 從共用的 queue 拉取 job、以自己的速度處理它們，並在完成後更新 job 的狀態。「async」的部分意思是，原始的 HTTP 請求在工作完成之前就立刻結束了。「worker pool」的部分，指的是專門用來執行背景 job 的那群 process。

Queue 在你的 web server 和 worker 之間充當緩衝。它持久化地儲存 job，這樣就算 worker 崩潰也不會遺失任何東西。常用的 queue 選項包括 Redis（搭配 Bull 或 BullMQ）、AWS SQS、RabbitMQ，或是在更高規模時使用 Kafka。Queue 也讓你能看到有多少 job 在等待、它們花了多長時間、以及哪些失敗了。

這種分離解決了我們之前看到的擴展問題。你的 web server 不再需要昂貴的 GPU，只因為某些請求會觸發影片處理。你可以用小型、便宜的 instance 運行 web server，讓影片 worker 使用 GPU 優化的機器。當月底的報告需求讓 PDF 產生備援，你可以多開幾台 worker，而不需要動到你的 web server 群。每個元件根據自身的具體需求來擴展。

bullbfnoat.org



用戶體驗也大幅改善。不是盯著一個凍住的瀏覽器，用戶會立刻收到他們的請求已被接收的確認。他們可以稍後再回來查看自己在 queue 中的位置或處理進度。當 job 完成，透過電子郵件、push notification 或 WebSocket 更新通知他們。他們可以自由地離開頁面、關掉筆電，等工作真的完成了再回來。

這個模式廣泛適用於任何超過幾秒的操作，例如圖片處理、影片轉碼、批量資料匯入、有嚴格流量限制的第三方 API 呼叫、報告產生或電子郵件行銷。具體的 queue 技術和 worker 實作可能有所不同，但核心模式是一樣的：快速接受、非同步處理、完成時通知。

取捨

管理長時間執行任務不是萬靈丹。這個方式解決了某些問題，但也製造了其他問題。

你得到的

快速的用戶回應時間：API 呼叫在毫秒內回傳，而不是在 30 秒後 timeout。用戶立刻得到他們的請求已被接收的確認。

獨立擴展能力：Web server 和 worker 分開擴展。在處理高峰期多加幾台 worker，不需要為閒置的 web server 付錢。

故障隔離：一台 worker 在處理某支影片時崩潰，不會拖垮你整個 API。失敗的 job 可以重試，不影響面向用戶的服務。

更好的資源利用：CPU 密集的 worker 跑在計算優化的 instance 上；記憶體需求高的任務得到高記憶體的機器；web server 使用便宜的通用型 instance。

你失去的

系統複雜度：你現在要管理 queue、worker 和 job 狀態追蹤。更多活動的元件，就意味著更多可能出問題的地方。

最終一致性（Eventual consistency）：API 回傳時，工作還沒完成。用戶可能在背景處理完成之前看到過時的資料。

buildmoat.org

Job 狀態追蹤：你需要基礎設施來儲存 job 狀態、處理重試，以及提供狀態查詢的 endpoint。這增加了 database 負載和 API 複雜度。

監控的 overhead：Queue 深度、worker 健康狀況、job 失敗率、處理延遲，你在監控的是一個分散式系統，而不是簡單的 request / response 循環。

Async 模式還製造了新的故障模式。Queue 滿了怎麼辦？如何處理一直讓 worker 崩潰的毒藥訊息（poison message）？什麼時候放棄重試一個失敗的 job？這些問題不是無法解決，但需要在設計時就先考慮到，這是同步系統不需要煩惱的。

如何實作

從本質上來說，這件事需要兩種核心技術：

Message Queue：一個讓你的系統不同部分之間能夠發送和接收訊息的系統。它是一種把訊息的發送方和接收方解耦的方式，讓它們能夠獨立運作。

Worker Pool：一群專門用來處理 queue 中訊息的 process。它們負責從 queue 拉取訊息、處理它們，並更新 job 狀態。

我們分別介紹每個元件，以及面試中你會討論到的常見技術選擇。

Message Queue

Queue 需要是持久化的（durable）。如果它崩潰，你不能遺失所有待處理的 job。它還需要處理來自多個 worker 的並發存取，而不會製造重複工作。以下是標準的選項：

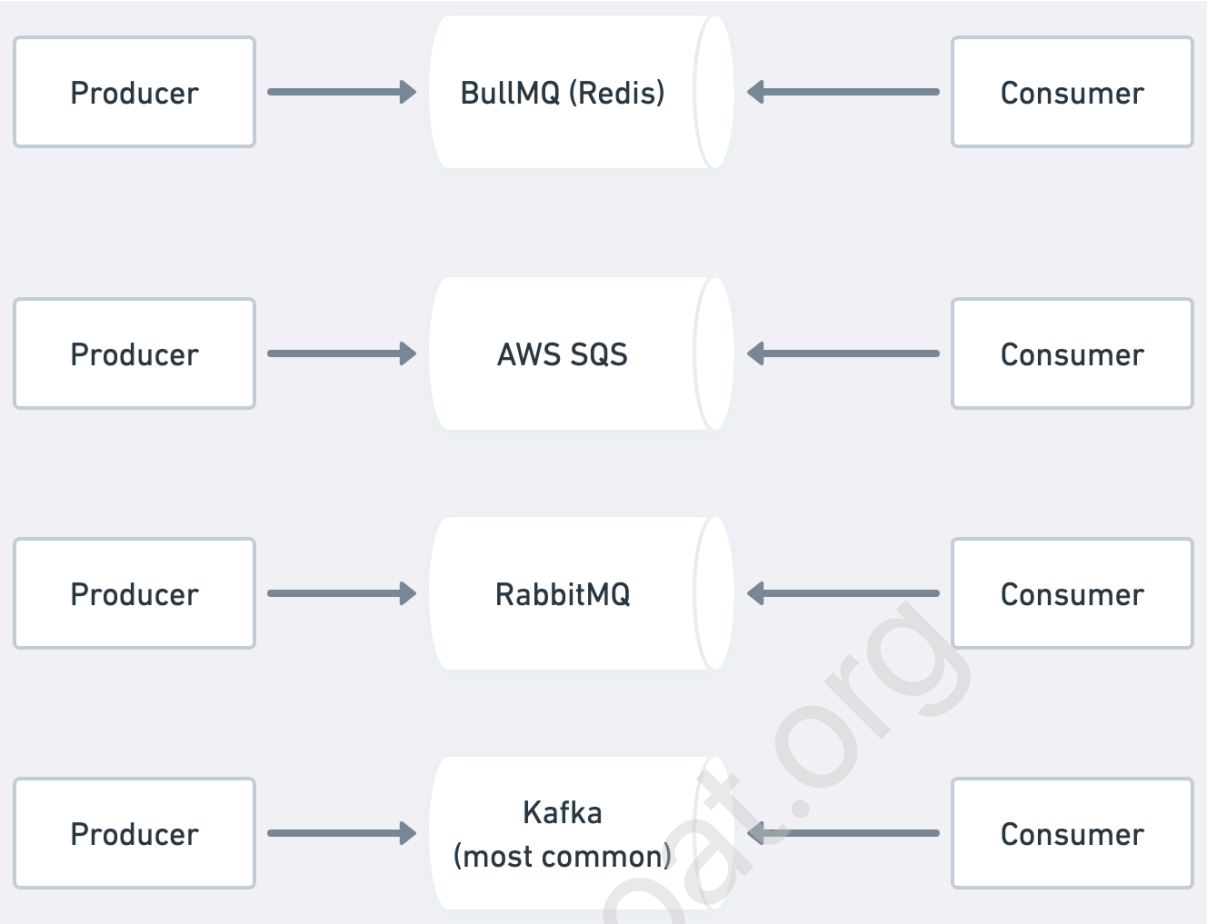
Redis with Bull / BullMQ 是很多 startup 的首選。Redis 提供儲存，Bull 在上面加上 job queue 的語義：自動重試、延遲 job、優先權 queue。設定簡單，90% 的使用案例都能應付。雖然 Redis 有持久化選項，但它本質上是 memory-first 的，所以在硬性崩潰時可能遺失 job。要真正的持久化，你會想用 SQS 或 RabbitMQ。

AWS SQS 完全省去了運維的 overhead，這很棒。Amazon 管理基礎設施、處理擴展，並保證訊息送達。按訊息計費，在低流量時很划算，但在大規模時成本可能變高。1MB 的訊息大小限制代表你通常還是要把 job 資料存在別的地方，只把 ID 透過 queue 傳遞。

RabbitMQ 給了你更多控制，支援複雜的路由模式，但需要自行架設。它在企業環境中久經考驗，能很好地處理複雜的 workflow。然而，運維負擔也很真實，你需要管理 cluster、處理升級，以及監控磁碟使用量。

Kafka 是首選，特別是當你的技術棧已經在用 event streaming 的時候。它的 append-only log 讓你可以重播訊息、fan-out 給多個 consumer、長時間保留資料。Kafka 還能處理超大流量，並在 partition 內保證嚴格的順序。

buildmoat.org



Worker

Worker 從 queue 拉取 job 並執行它們。關鍵的決策是如何運行這些 worker：

一般 server 是最簡單的方式。你在幾台機器上跑 10 到 20 個 worker process，每個都在一個 loop 裡不斷拉取 job。這讓你對環境有完全的控制，也讓除錯變得直觀，更可以 SSH 進去查看發生了什麼事。當你掌控基礎設施時，長時間執行的 job 不是問題。缺點是你在管理 server，並且在安靜時段也要為閒置的容量付錢。

```
# 簡化版的 worker loop
while True:
    job = queue.pop()  # 阻塞，直到有 job 可用
    if job:
        process_job(job)
        mark_complete(job.id)
```

Serverless function（Lambda、Cloud Functions）省去了 server 管理。每個 job 觸發一次 function 執行，自動擴展。對於流量高峰不穩定的工作負載特別適合，例如某分鐘處理 1,000 個 job，下一分鐘可能一個都沒有。你只為實際的執行時間付費。但限

buildmoat.org

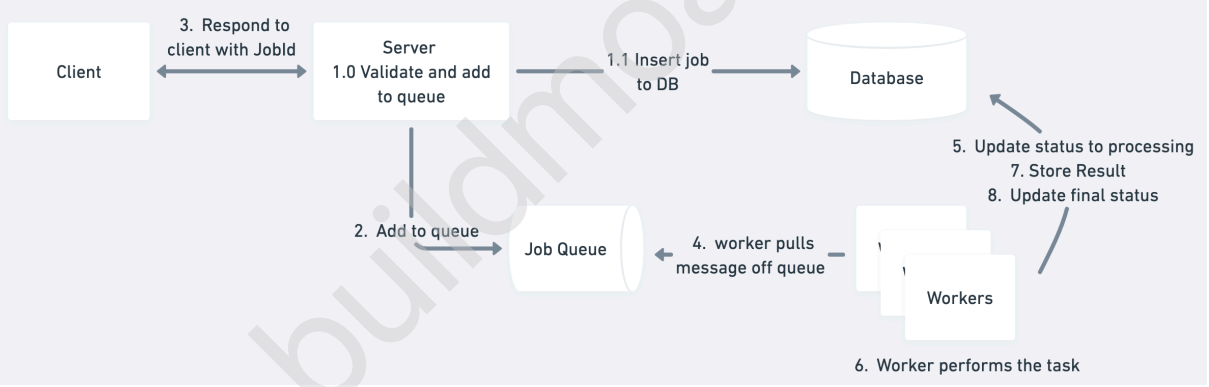
制是真實的：你受限於 15 到 60 分鐘的執行時間（取決於供應商）、cold start 增加延遲，以及極小的本地儲存空間。

Container-based worker（在 Kubernetes 或 ECS 上）提供了一個中間方案。你把 worker 打包成 Docker container，讓 orchestrator 處理擴展和部署。這個方式比純 server 複雜，但比 serverless 更有彈性。你仍然可以處理長時間執行的 job，同時透過自動擴展獲得更好的資源利用率。

把所有東西組合在一起

以下是所有東西協同運作的流程：

1. Web server 驗證請求，並在 database 裡建立一筆狀態為 `pending` 的 job 記錄
2. Web server 把包含 job ID 的訊息推進 queue（不是完整的 job 資料）
3. Web server 立刻把 job ID 回傳給 client
4. Worker 從 queue 拉取訊息，從 database 抓取 job 的詳細資料
5. Worker 把 job 狀態更新為 `processing`
6. Worker 執行實際的工作
7. Worker 儲存結果（檔案存 S3，metadata 存 database 等）
8. Worker 把 job 狀態更新為 `completed` 或 `failed`



buildmoat.org

重要的是，每個元件可以獨立失敗，系統整體仍能正常運作。如果 queue 掛掉，web server 仍然接受請求，job 在 database 裡以 `pending` 狀態堆積。當 worker 崩潰，job 留在 queue 裡，等新的 worker 上線後繼續處理。如果 database 有問題，worker 可以持續重試，直到它恢復。

在面試中，不要把這件事搞得太複雜。除非有特殊理由，否則直接選 Kafka（或你熟悉的 queue）。Worker 就用一般的 server process，除非面試官特別要求 serverless。重點在於展示你理解關注點分離（separation of concerns），而不是辯論各種 queue 技術的優劣。

什麼時候在面試裡用這些

不要等面試官來問。關鍵是識別那些呼喊著需要 async 處理的問題，然後主動提出建議。

以下是幾個常見的信號，應該觸發你對管理長時間執行任務的思考：

慢速操作

如果聽到「影片轉碼」、「圖片處理」、「PDF 產生」、「批量發送電子郵件」或「資料匯出」，那就是你的提示。這些操作需要幾秒到幾分鐘。立刻說：「影片轉碼要花好幾分鐘，所以我會立刻回傳一個 job ID，然後非同步地處理它。」

數字不合理

例如「我們每天處理 100 萬張圖片」，而你知道圖片處理需要 10 秒，就大聲算出來：「那大概是每秒 12 張，每張要 10 秒，也就是每過 1 秒就累積 120 秒的工作量——至少需要 120 個 worker 才跟得上。我們沒辦法在 web server 上跑這個——它們會完全被圖片處理塞滿，無法服務用戶請求。我會把這個 offload 到一個專用的 async worker pool，讓 web 層保持快速，worker 可以在計算優化的硬體上獨立擴展。」

不同操作需求不同硬體

如果問題同時涉及簡單的 API 請求和 GPU 密集的工作（像 ML inference 或影片處理），那是明確的 async 信號。「我們不應該把 GPU 工作負載跑在處理登入請求的同一台 server 上。我會用跑在 GPU instance 上的 async worker 把這兩件事分開。」

規模與故障考量

面試官詢問「如果 server 在處理到一半時崩潰怎麼辦？」或「如何處理 10 倍的流量？」這類問題是引入 async worker 的絕佳開場白。「有了 async worker，如果其中一台在處理 job 到一半時崩潰，另一台 worker 會從 queue 接手那個 job。沒有任何用戶請求會遺失。」

關鍵技能是主動出擊。你應該是那個指出某個操作花太長時間的人，而不是等他們來問。這展示了系統設計的成熟度——你在他們提出這些顧慮之前，就已經在思考 timeout、資源利用率和用戶體驗了。

具體範例

以下是管理長時間執行任務模式在常見系統設計面試題中出現的樣子：

YouTube / 影音平台：影片上傳觸發多個 async job：轉碼成不同解析度（1080p、720p、480p）、產生縮圖、提取字幕、執行內容審核。每部影片可能需要多個 worker 跑好幾個小時。

buildmoat.org

Instagram / 圖片分享：照片上傳產生 async 任務，包括產生多種尺寸、套用濾鏡、提取 metadata、執行圖片識別來加標籤，以及審核內容政策。Fan-out 問題也用到 async worker：當一個有幾百萬追蹤者的人發文，worker 負責把那篇貼文傳播到所有追蹤者的 feed。

Robotaxi / 共乘：配對乘車是非同步執行的。用戶看到「正在尋找附近的自駕車……」，同時 worker 在評估自駕車的可用性、計算最佳路線、確定定價。自駕車的位置更新也透過 async worker 處理，在不過載主要 API 的情況下更新乘客的 App。

Stripe / 支付處理：雖然基本的信用卡授權是同步的，但詐欺偵測、3D Secure 驗證和 webhook 通知都由 async worker 處理。對於需要延遲確認的支付方式（銀行轉帳、某些地區性方式），API 回傳 `pending` 狀態，worker 在背景處理實際的清算。

Dropbox / 檔案同步：檔案上傳觸發 async 處理：病毒掃描、搜尋索引、產生預覽，以及同步到其他裝置。大型檔案可能需要幾分鐘處理，但上傳的 API 立刻回傳。

注意這個模式：任何涉及大量計算、外部 API 呼叫，或 fan-out 給多個用戶的操作，都能從 async 處理中受益。如果面試官提到這些情境中的任何一個，管理長時間執行任務應該是你解法的一部分。

常見的 Deep Dive 問題

以下是五個常見的深挖方向，以及如何應對。你可能會主動提起它們，也可能被問到。

處理故障

面試官可能先問一個基本問題：「如果 worker 在處理 job 的過程中崩潰了怎麼辦？」如果我們遵循這個模式，答案其實很簡單：那個 job 會被另一個 worker 重試。

但我們怎麼知道 worker 崩潰了？通常你會有一個心跳機制（heartbeat mechanism），定期向 queue 回報 worker 還活著。如果 worker 沒有回報，queue 就會假設它掛了，然後重試那個 job。

心跳的間隔是個關鍵的設計決策。如果太長，崩潰會讓 job 延遲更久（queue 會樂觀地假設 worker 還活著，比實際情況多等很長時間）。如果太短，你會向 queue 發送很多不必要的訊息，更糟的是，你可能把仍在執行中的 job 標記為失敗（例如，遇到 garbage collection pause 的情況）。

每個 queue 系統處理這件事的方式不同。比如在 SQS 裡，你可以設定 visibility timeout；在 RabbitMQ 裡，你可以設定 heartbeat interval；在 Kafka 裡，你可以設定 session timeout。選擇在你的用戶或 client 能接受的額外延遲範圍內，盡可能長的間隔。對大多數系統而言，10 到 30 秒是個好的起點。

buildmoat.org

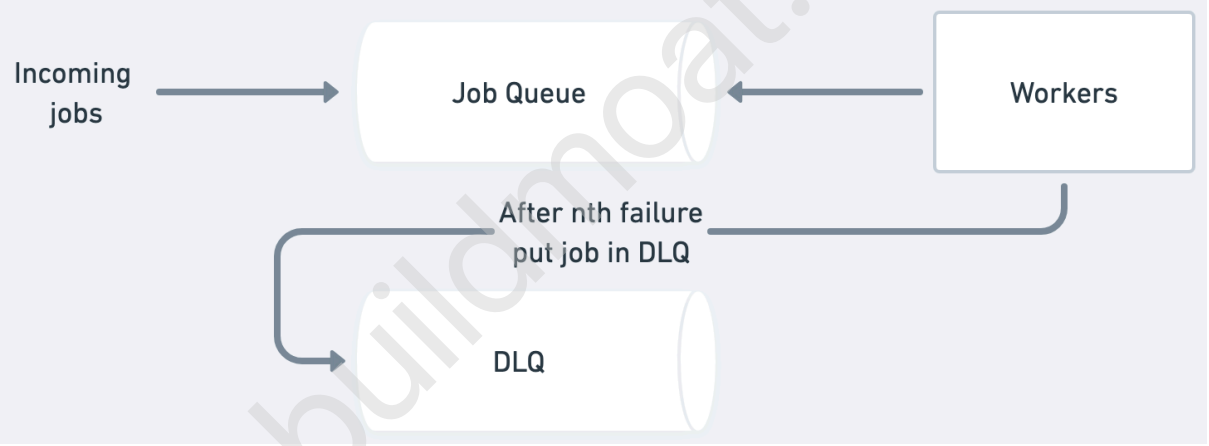
處理反覆失敗

面試官問：「如果一個 job 一直失敗怎麼辦？也許你的程式碼有 bug，或者輸入的資料有問題，每次都讓 worker 崩潰。」

如果不處理，失敗的 job 就會被無限重試。你的 worker 在那些注定失敗的 job 上浪費資源，而有效的工作在 queue 裡越堆越多。更糟的是，一個讓 worker 崩潰的毒藥訊息（poison message）可能讓整個 worker 群掛掉，因為每台 worker 試著處理它都會死掉。

解法是**Dead Letter Queue（DLQ，死信佇列）**。一個 job 失敗達到一定次數（通常是 3 到 5 次）後，把它移到一個獨立的 queue，而不是繼續重試。這把有問題的 job 隔離出來，讓健康的工作繼續。你的 DLQ 就變成了一批需要人工調查的 job，可能是程式錯誤需要修正，或者有資料要清理。修好之後，你可以把 job 移回主要的 queue 重新處理。

大多數 queue 系統都有內建的 DLQ 支援。在 SQS 裡，你設定一個 redrive policy；在 RabbitMQ 裡，你設定一個 dead letter exchange。重點是監控你的 DLQ，如果 dead letter queue 持續增加，通常意味著你的系統裡有個需要立刻處理的 bug。



buildmoat.org

防止重複工作

面試官繼續問：「一個用戶不耐煩，點了三次『產生報告』。現在 queue 裡有三個一模一樣的 job。你怎麼防止做重複的工作？」

如果不做去重，你就在浪費資源處理一樣的 job。更糟的是，你可能發三封電子郵件給同一個用戶，或者把他的信用卡刷了三次。當 job 不是冪等的（idempotent），副作用會不斷疊加。

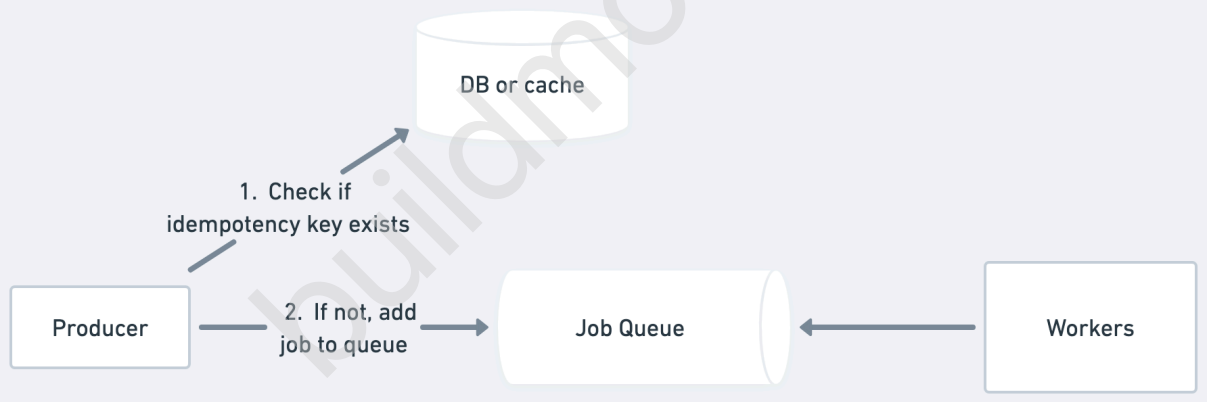
解法是**idempotency key（冪等鍵）**。在接受一個 job 時，要求一個代表這個操作的唯一識別碼。對用戶發起的操作，把 user ID + action + timestamp（可能取整到你想要防止重複工作的時間長度）組合起來。對系統產生的 job，用基於輸入資料的確定性

ID。在開始工作之前，先確認有沒有這個 key 的 job 已經存在。如果有，就回傳現有的 job ID，而不是建立一個新的。

```
def submit_job(user_id, job_type, job_data, idempotency_key):
    # 確認 job 是否已存在
    existing_job = db.get_job_by_key(idempotency_key)
    if existing_job:
        return existing_job.id

    # 建立新的 job
    job_id = create_job(user_id, job_type, job_data)
    db.store_idempotency_key(idempotency_key, job_id)
    queue.push(job_id)
    return job_id
```

你還會希望讓工作本身也是冪等的。如果一個 job 執行到一半失敗然後被重試，重新執行應該是安全的。這可能意味著在繼續之前，先確認電子郵件是否已經發送，或者檔案是否已經被處理過。



管理 Queue Backpressure

下一個問題：「黑色星期五到了，你突然收到比平時多 10 倍的 job。你的 worker 跟不上。Queue 長到幾百萬個待處理的 job。你怎麼辦？」

當 worker 處理 job 的速度跟不上進來的速度，queue 就會無限成長。記憶體使用量爆炸。Job 的等待時間拉長到幾個小時。新的 job 因為 queue 已滿而被拒絕。用戶越來越沮喪，等待的結果遲遲不來。

解法叫做 **backpressure（背壓）**：當 worker 不堪重負時，放慢 job 的接受速度。你可以設定 queue 深度上限，當 queue 太深時拒絕新的 job，立刻回傳「系統忙碌中」的回應，而不是接受你根本無法處理的工作。

buildmoat.org

你還應該根據 queue 深度來自動擴展 worker。當 queue 超過某個閾值，就多開 worker；縮短了就縮減。在 AWS 上，CloudWatch alarm 加上 Auto Scaling group 讓這件事很直觀。關鍵 metric 是 queue 深度，而不是 CPU 使用率，如果等到 CPU 飆高，你的 queue 早就已經累積待處理的事件了。

處理混合工作負載

面試官繼續問：「你的某些 PDF 報告要 5 秒，但年底的報告要 5 個小時。它們全都在同一個 queue 裡。這會造成什麼問題？」

長的 job 會卡住短的 job。一個用戶請求一份簡單的報告，卻要在某人的大型年末匯出後面等好幾個小時。Worker 的利用率變得不均衡，某些 worker 處理幾百個快速 job，而其他 worker 卡在單一個長 job 上。當你無法預測 job 需要多長時間時，autoscaling 就變成噩夢。

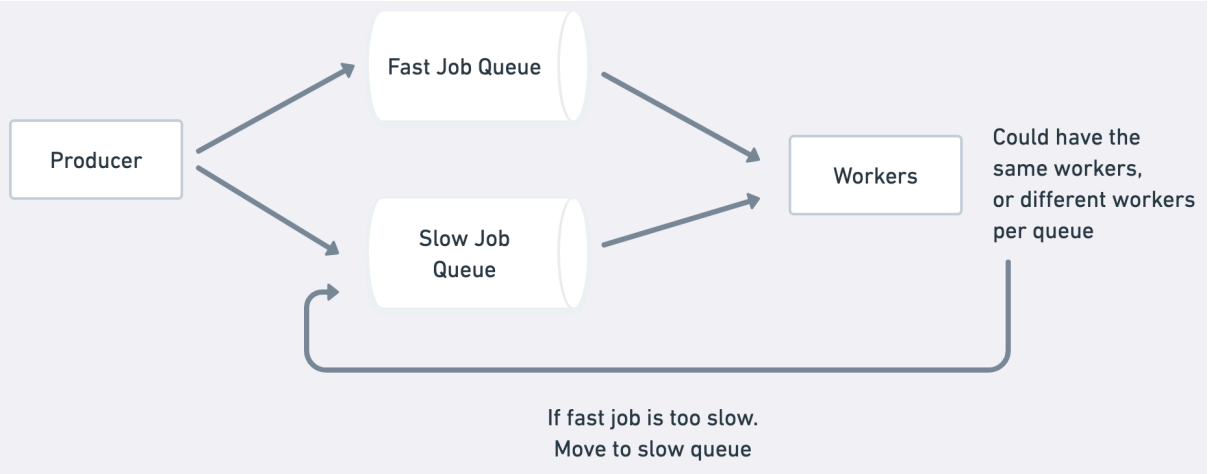
解法是**按 job 類型或預期執行時間分別建立 queue**。快速報告進入有很多 worker 的「fast」queue，複雜報告進入有較少但更強力 worker 的「slow」queue。這防止了隊首阻塞（head-of-line blocking），讓你能獨立調校每個 queue。另一個選項是把大型 job 拆成更小的 chunk，使用同樣的 queue 基礎設施，像是把 news feed 的 fan-out 拆成幾批追蹤者分批處理。

```
queues:
  fast:
    max_duration: 60s
    worker_count: 50
    instance_type: t3.medium

  slow:
    max_duration: 6h
    worker_count: 10
    instance_type: c5.xlarge
```

在 job 提交時路由到對應的 queue。如果無法事先預測執行時間，就先把 job 放進 fast queue，如果超過時間限制就移到 slow queue。這讓你的 fast queue 保持快速回應，同時給長時間執行的 job 它們需要的資源。

buildmoat.org



協調有依賴關係的 Job

最後一個問題：「如果產生一份報告需要三個步驟：抓取資料、產生 PDF、然後發電子郵件，怎麼處理有依賴關係的 job？」

如果沒有適當的 orchestration，你最終會得到義大利麵程式碼。Worker 完成第一步後直接 queue 第二步，讓整個流程難以理解和修改。錯誤處理也變得困難，第二步失敗了，你是只重試那一步，還是從頭來過？要搞清楚哪一步失敗了、為什麼，需要翻遍 log。

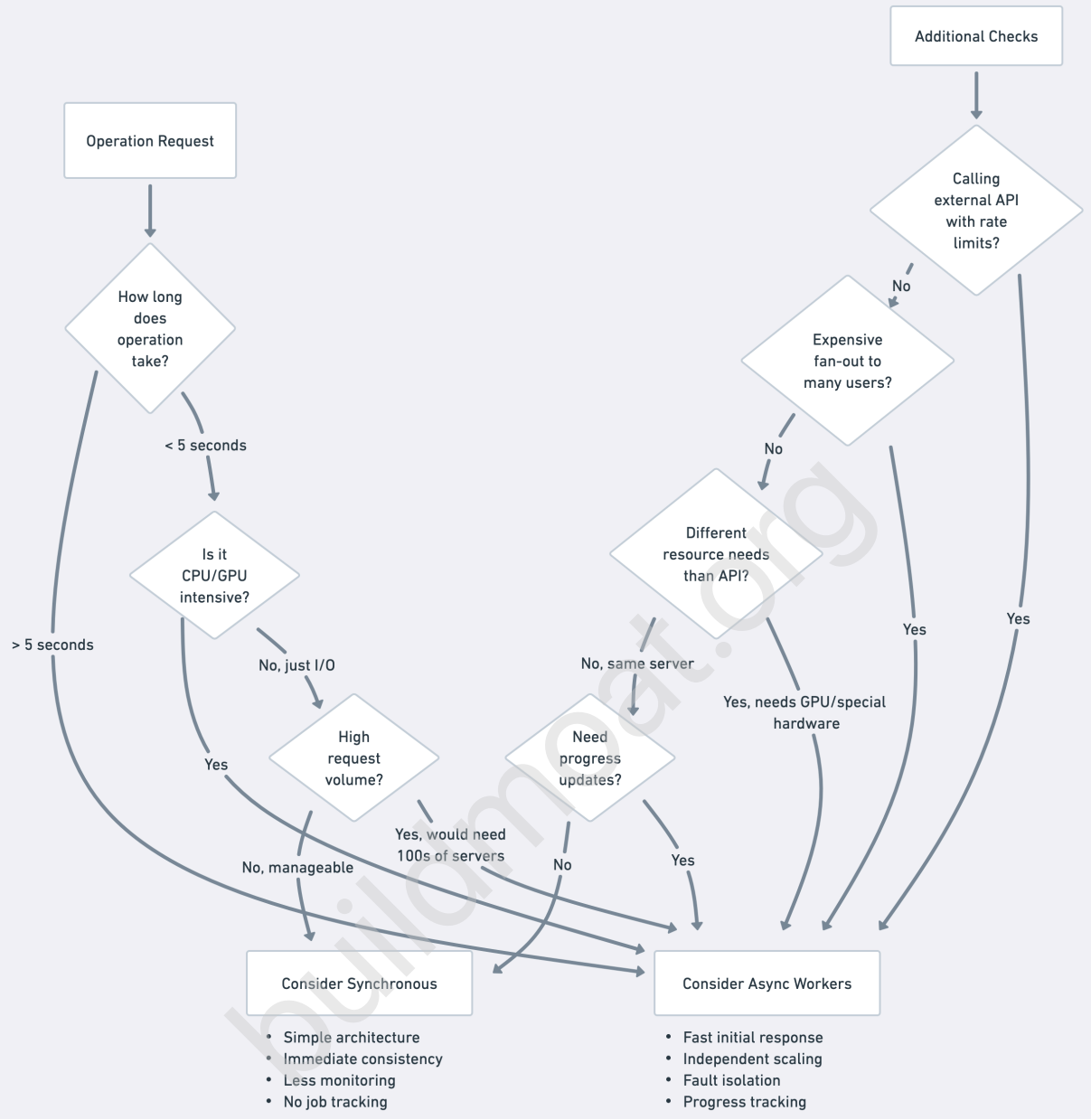
對於簡單的鏈式操作，讓每個 worker 在標記自己完成之前，先把下一個 job 加進 queue。在每個 job 裡帶入完整的 context，這樣每個步驟都可以獨立重試：

```
{
  "workflow_id": "report_123",
  "step": "generate_pdf",
  "previous_steps": ["fetch_data"],
  "context": {
    "user_id": 456,
    "data_s3_url": "s3://bucket/data.json"
  }
}
```

對於有分支或平行步驟的複雜 workflow，使用像 AWS Step Functions、Temporal 或 Airflow 這樣的 workflow orchestrator。這些工具讓你把 workflow 定義成程式碼、按步驟處理重試，並提供 workflow 在哪裡卡住的可見性。代價是額外的複雜度，因此只有在 job 的依賴關係真的很複雜時，才去用 orchestration。

buildmeat.org

總結



管理長時間執行任務的模式，是系統設計面試中非常常見的一個。一旦你認出那些信號——超過幾秒的操作、資源需求不匹配，或者怎麼算都對不上的規模需求，解法就會自然浮現：把工作加進 queue、立刻回傳、非同步處理。

記住，在面試中，你要是那個主動發現這些問題的人，而不是等面試官來指出它們。當他們提到影片上傳或 PDF 產生，立刻說「那個對同步請求來說太慢了，所以我會用 async worker」。從一開始就展示你在思考 timeout、資源利用率和用戶體驗。

實作細節的重要性，比不上展示你理解這個模式。選一個你能自在討論的 queue 技術（Kafka 是個安全的預設選項），提到持久性和複雜度的取捨，並準備好應對圍繞故障處理和擴展的常見深挖問題。

最重要的是，不要想太多。這個模式在幾乎所有有規模的公司都會用到，從 Robotaxi 的乘車配對到 Instagram 的 feed fan-out。掌握這個模式，你就能自信地應對一大類系統設計問題。

buildmoat.org